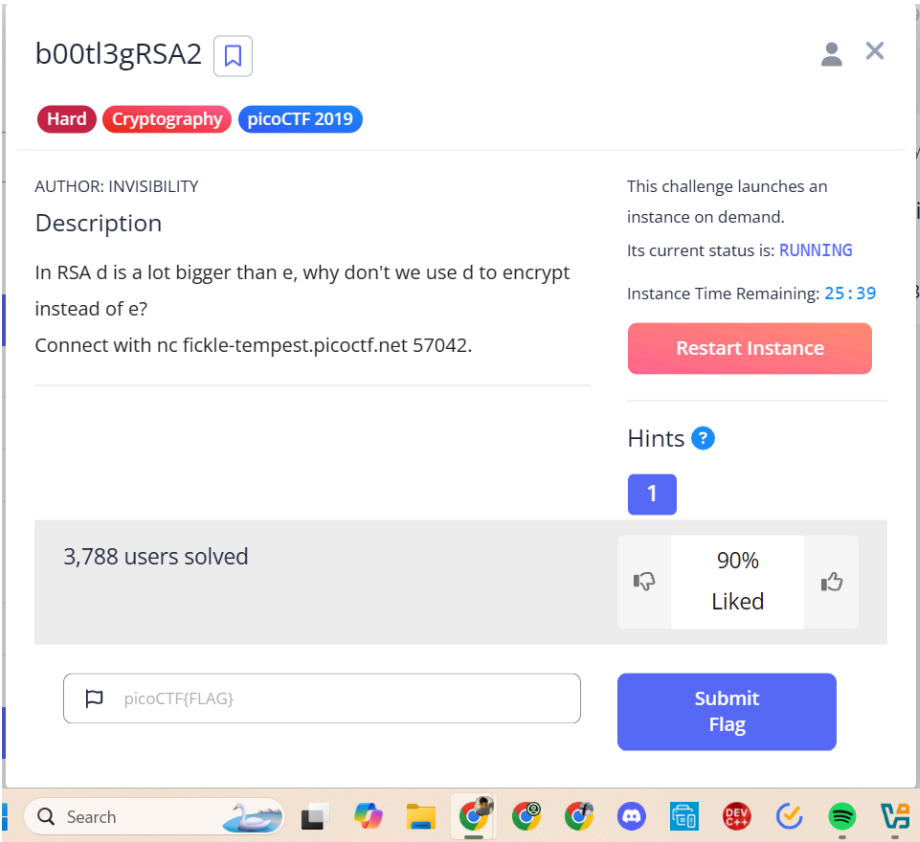


PRACTICAL ACTIVITY 2 [CHALLENGE 2]: b00tl3gRSA2 Write-up



CHALLENGE TITLE: **b00tl3gRSA2**

(<https://play.picoctf.org/practice/challenge/2?category=2&page=1&search=b00tl3gRSA2>)

PLATFORM: **picoCTF**

AUTHOR: **invisibility**

CATEGORY: **Cryptography (Hard)**

TOOL USED: **Kali Linux**

Challenge Overview:

The challenge description states:

In RSA d is a lot bigger than e, why don't we use d to encrypt instead of e?

This hint suggests that the implementation of RSA in the challenge might be misusing the private exponent (d) during encryption. Normally, RSA encryption uses the public exponent (e) while the private exponent (d) is reserved for decryption. If these roles are accidentally reversed, the system may become vulnerable.

The challenge instance provided three values after connecting:

- **c** – ciphertext
- **n** – RSA modulus
- **e** – public exponent

The goal was to recover the original plaintext (the flag).

PROCESS:

To solve the b00tl3gRSA2 challenge from picoCTF, I first connected to the challenge instance using the terminal in Kali Linux. After establishing the connection, the server provided three values: the ciphertext *c*, the modulus *n*, and the exponent *e*.

```
jure@kali: ~
Session Actions Edit View Help

(jure@kali)-[~]
$ nc fickle-tempest.picoctf.net 57042
c: 90714378833210105727683761418805928753013371505135142130360426308815728937
50037833208825730859356222883368660024641652319327759422193417263939110073451
11051899607984638591275118214617581306523931295837823878828358429023817156435
60232075315804945989437291190870776172161509598822535472228339018248661720467
111
n: 10429533324384323857008590424255268910111867998061580284917826600188852284
94289144299130696987788466377189891815048606829659389395716819241687108747984
09489338734014582722714356052703582181085521666650265787172922847402957218722
62283715184220208856204759167286573369595344096551271128236022382140363436368
5117
e: 19154044752168961341372872476057252636237002490908827121972467607591593466
52618237146091073583627871479600460485820991470738729384446592976936088147265
90531660204848772405652124340628536050951290218950458518490246247407320572764
15955076761494577857378330380274784848400073116662494908519116610131646721067
393
```

From the challenge description, which hints that RSA uses a much larger *d* than *e*, I inferred that the encryption might have been implemented incorrectly by using the private exponent *d* instead of the public exponent *e*. Normally in RSA, encryption is performed using $c = m^e \bmod n$, but if the system mistakenly encrypts with *d*, the ciphertext would be generated as $c = m^d \bmod n$. Because of the mathematical relationship between the exponents in RSA, raising the ciphertext to the power of *e* modulo *n* can recover the original plaintext. After observing that the value of *e* was 65537, a commonly used RSA public exponent, I computed $m = c^{65537} \bmod n$ using modular exponentiation in the terminal.

```
jure@kali: ~
Session Actions Edit View Help

(jure@kali)-[~]
$ ^C
(jure@kali)-[~]
$ python
Python 3.13.9 (main, Oct 15 2025, 14:56:22) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license()" for more information.
>>> pow(90714378833210105727683761418805928753013371505135142130360426308815\
728937500378332088257308593562228833686600246416523193277594221934172639391\
0073451110518996079846385912751182146175813065239312958378238788283584290238\
1715643560232075315804945989437291190870776172161509598822535472228339018248\
661720467111, 65537, 1042953332438432385700859042425526891011186799806158028\
4917826600188852284942891442991306969877884663771898918150486068296593893957\
1681924168710874798409489338734014582722714356052703582181085521666650265787\
1729228474029572187226228371518422020885620475916728657336959534409655127112\
82360223821403634363685117)
180638594769037903267909311328535969949661653394344568289572221
>>> hex(pow(9071437883321010572768376141880592875301337150513514213036042630\
8815728937500378332088257308593562228833686600246416523193277594221934172639\
3911007345111051899607984638591275118214617581306523931295837823878828358429\
0238171564356023207531580494598943729119087077617216150959882253547222833901\
8248661720467111, 65537, 104295333243843238570085904242552689101118679980615\
8028491782660018885228494289144299130696987788466377189891815048606829659389\
3957168192416871087479840948933873401458272271435605270358218108552166665026\
5787172922847402957218722622837151842202088562047591672865733695953440965512\
711282360223821403634363685117))
'0x7869636f4354467b6261645f31643361355f333830313235357d'
```

The output produced a large hexadecimal value that appeared to represent encoded text. I then converted this hexadecimal value into ASCII, which revealed the readable message containing the challenge flag. This process successfully solved the challenge by exploiting the incorrect use of RSA exponents in the encryption step.

Kali Linux [Running] - Oracle VirtualBox

FileMachineViewInputDevicesHelp

Hex to String | Hex to ASCII

www.rapidtables.com/convert/number/hex-to-ascii.html

OffSecKali LinuxKali ToolsKali DocsKali ForumsKali NetHunterExploit-DBGoogle Hacking DB

From

Hexadecimal

To

Text

Open File

Sample

Paste hex code numbers or drop file

7069636f4354467b6261645f31643361355f333830313235357

Character encoding

ASCII

= Convert

× Reset

↕ Swap

picoCTF{bad_1d3a5_3801255}

Copy

Save

Text to hex converter

CalendarAlt4/Cryptography/WEEK 3picoCTF - picoGym Challengeshow to connect picocTF challenge

play.picoctf.org/practice/challenge/2?category=2&page=1&search=rsa

Hide Solved

Show Bookmarked

Show Assigned

Search by Name

rsa

Difficulty

All Difficulties

Easy

Medium

Hard

Category

All Categories

Web Exploitation

Cryptography

Reverse Engineering

Even RSA can be broken???

rsa_oracle

Mini RSA

b00t13gRSA2

HardCryptographypicoCTF 2019

AUTHOR: INVISIBILITY

Description

In RSA d is a lot bigger than e, why don't we use d to encrypt instead of e?

Connect with nc fickle-tempest.picoctf.net 57042.

This challenge launches an instance on demand.

Its current status is: RUNNING

Instance Time Remaining: 25:39

Restart Instance

Hints

1

3,788 users solved

90% Liked

picoCTF{bad_1d3a5_3801255}

Submit Flag

91 solves86%

ptographyHard

MiniRSA

91 solves87%

>_ Webshell

29°C Windy

ENG US4:02 PM3/13/2026

Hurray! You solved this challenge.

Reflection:

This challenge helped me better understand how critical proper implementation is in cryptography. While algorithms like RSA are mathematically secure, even a small mistake—such as using the private exponent d for encryption instead of the public exponent e —can completely break the system's security. Solving this problem showed me that many cryptographic vulnerabilities do not come from weak algorithms but from incorrect usage or configuration. Through this exercise on picoCTF using Kali Linux, I also improved my practical skills in analyzing encryption outputs, performing modular exponentiation, and converting encoded data into readable formats. Overall, the challenge reinforced the importance of understanding both the theory and the real-world implementation of cryptographic systems.